



Build your IT Skill

1. Introduction
2. Project Setup
3. Basic syntax
4. Database
5. Image upload
6. Spring Security



spring Boot Features



រៀបរាងដោយ៖
គ្រូអោយពីចិត្តល្អ
អនុបណ្ឌិត RUPP, TKU
(Software Engineering)
Assistands:
សៀន ធានីក

២០២៦

Introduction

Spring Framework គឺជា Framework មួយនៃភាសា Java ដែលប្រើសម្រាប់ អភិវឌ្ឍន៍កម្មវិធីផ្សេងៗដូចជា Web Application, API, Backend Service, Enterprise Application, Cloud , AI និងច្រើនទៀត។



I. Core Spring Framework

Core Spring framework គឺជាគោលគំនិត ឬ លក្ខណៈសំខាន់នៅក្នុង Spring Framework មាន 4 ចំណុចសំខាន់ៗដូចជា IoC(Inversion Of Control), DI(Dependencies Injection), Spring Container និង Bean។

- IoC : Spring ជាអ្នកគ្រប់គ្រង object
- DI : ការដាក់ injection object មួយ ទៅក្នុង object មួយទៀត
- Spring Container : core component មួយនៃ Spring Framework ដែលមានភារកិច្ចក្នុងការ create, configure, manage និង inject dependencies របស់ objects (Beans) ក្នុង application។
- Bean : ជា object មួយដែលត្រូវបាន create, manage និង control ដោយ Spring Container នៅក្នុង application។

II. Spring Boot

Spring Boot គឺជា Framework មួយដែលស្ថិតនៅលើ Spring Framework ឬជា Implementation(extend)ពីលើ Spring Framework ដែលធ្វើឲ្យការអភិវឌ្ឍ Spring Application ឬសរសេរកម្មវិធីមានភាពងាយស្រួលជាងមុន និង Config តិចជាងមុន។ ប៉ុន្តែវាមិនមានជំនួស Spring Framework ទេគ្រាន់តែ Implement លើ Spring core, Spring MVC, Spring API, Spring Security ជាដើម។



Spring Framework Vs Spring Boot

Spring Framework

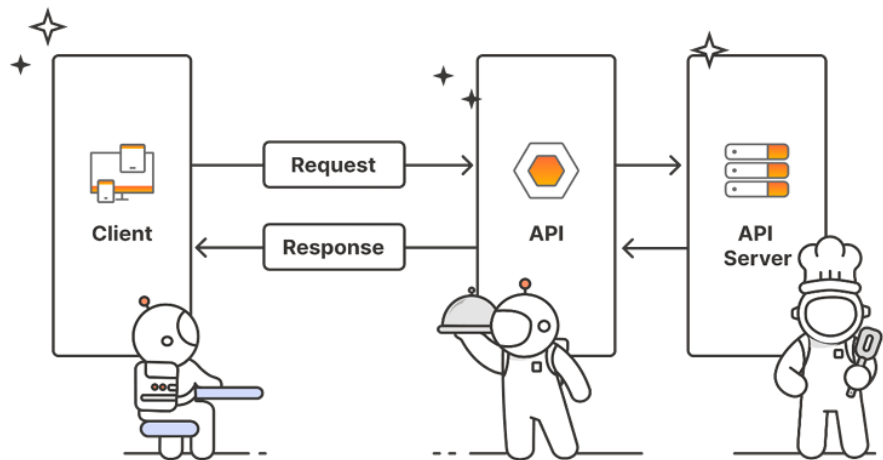
- ត្រូវ config ដោយដៃ (XML ឬ Java Config)
- ត្រូវការប្រើ External Server (Tomcat, Jetty, Apache....)
- ពេល setup project ត្រូវ configuration dependency ដោយដៃ
- គ្រប់គ្រង Dependency ដោយដៃ
- ការសរសេរមានភាពស្មុគស្មាញ

Spring Boot

- Auto Configuration (កាត់បន្ថយការសរសេរកូដច្រើន)
- មាន Embedded Server (Tomcat, Jetty, Apache....) ស្រាប់ៗ
- ប្រើ Starter dependencies (spring-boot-starter-*)
- មាន starter POM.xml ជួយគ្រប់គ្រង
- ការសរសេរមានភាពងាយស្រួលក្នុងការចាប់ផ្តើមចំពោះអ្នកថ្មី

III. API

API (Application Programming Interface) ជាវិធីសាស្ត្រមួយដែលអនុញ្ញាតឱ្យ Application មួយអាចទាក់ទងទៅនឹង Application មួយទៀតបាន និងធ្វើការផ្លាស់ប្តូរទិន្នន័យគ្នាទៅវិញទៅមកបានតាម HTTP Protocol។



ចំពោះចំណុចរបស់API:

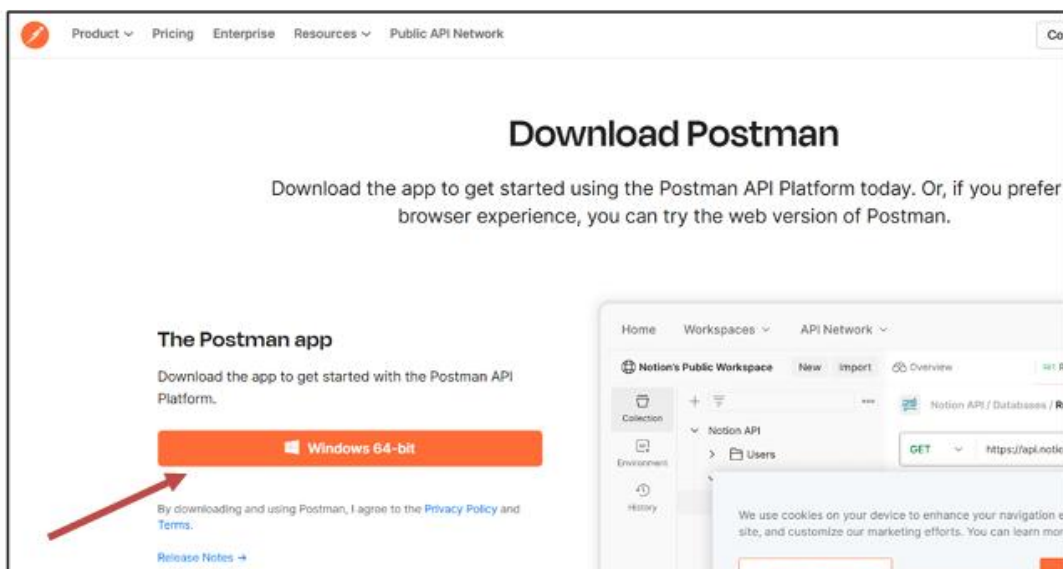
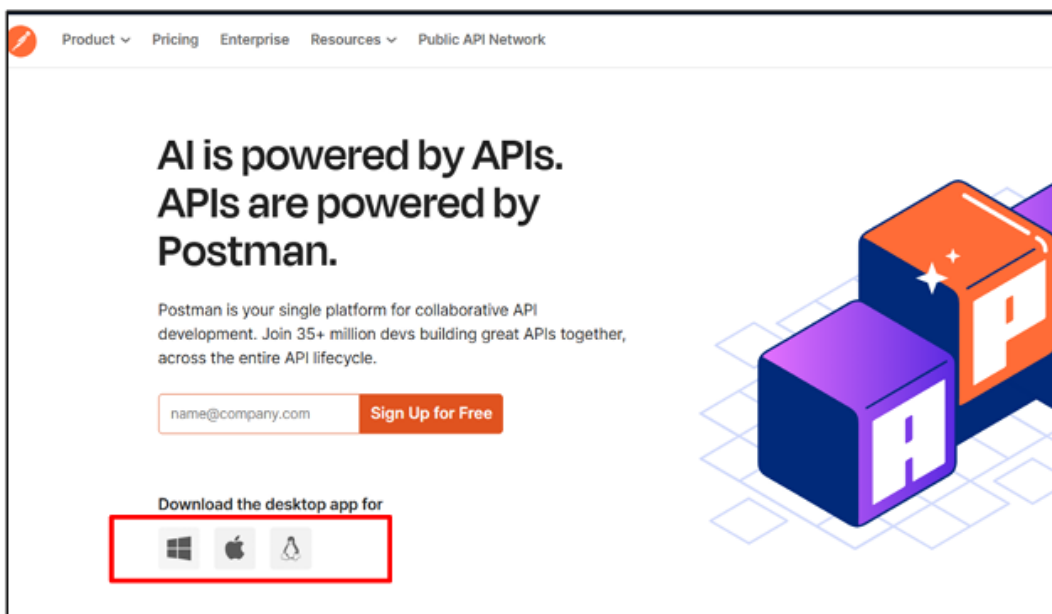
- Endpoints: គឺជា URL ដែលកំណត់សម្រាប់ API requests អាចប្រើប្រាស់បាន។
- Api ប្រើនៅលើ HTTP Requestដែលជាទូទៅគេប្រើលើ HTTP request ដូចជា ៖
 - . GET : ទាញយកទិន្នន័យ
 - . POST : បញ្ចូលទិន្នន័យ
 - . PUT : កែប្រែទិន្នន័យ
 - . PATCH : កែប្រែទិន្នន័យ ដូចគ្នា
 - . DELETE : លុបទិន្នន័យ

សម្រាប់ Tool ដែលអាចTesting លើ API ដែលបង្កើតឡើងដោយការសរសេរ Spring Boot Application បានមានដូចជា ៖ Postman, Bruno, Swagger....។ ដោយ Postman ពេញនិយមក្នុងការ Test លើ API របស់លោកអ្នក។

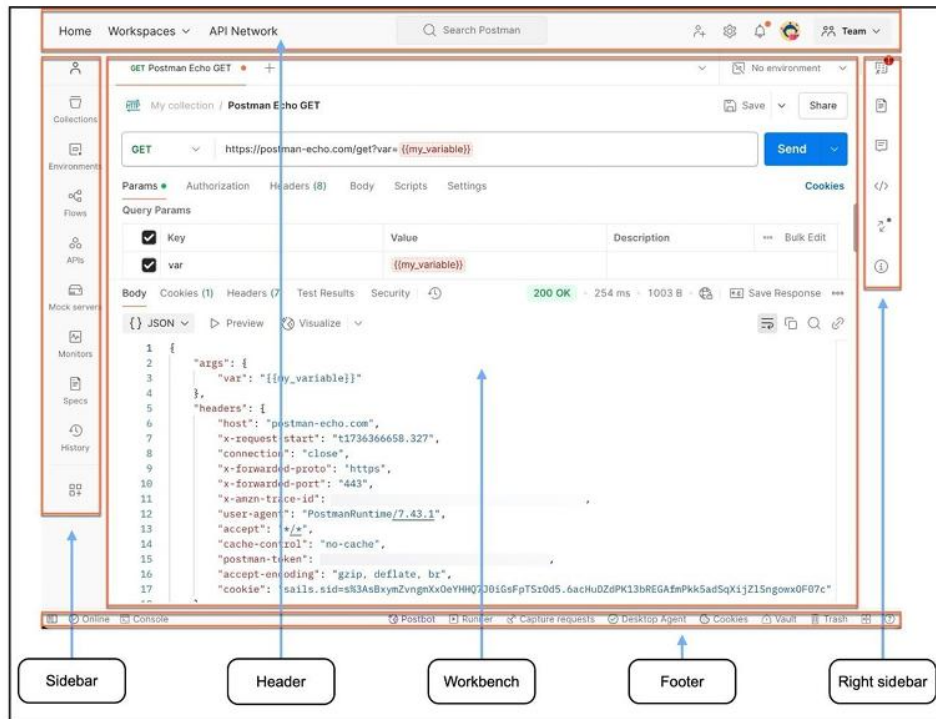
. Postman

Postman គឺជា tool សម្រាប់ testing API ដែលមានប្រើប្រាស់ច្រើនក្នុងអាជីវកម្ម ដើម្បីធ្វើឲ្យមានភាពងាយស្រួលក្នុងការទំនាក់ទំនង requests នឹង API និង response data មកវិញបានតាមGUI (Graphical User Interface) ដែលមានភាពងាយស្រួល សម្រាប់ធ្វើការបញ្ជូនសំណើ API, កំណត់អាចយល់ស្រួលក្នុង testing, debug, និង document API endpoints។

ជំហានក្នុងការ install postman ៖

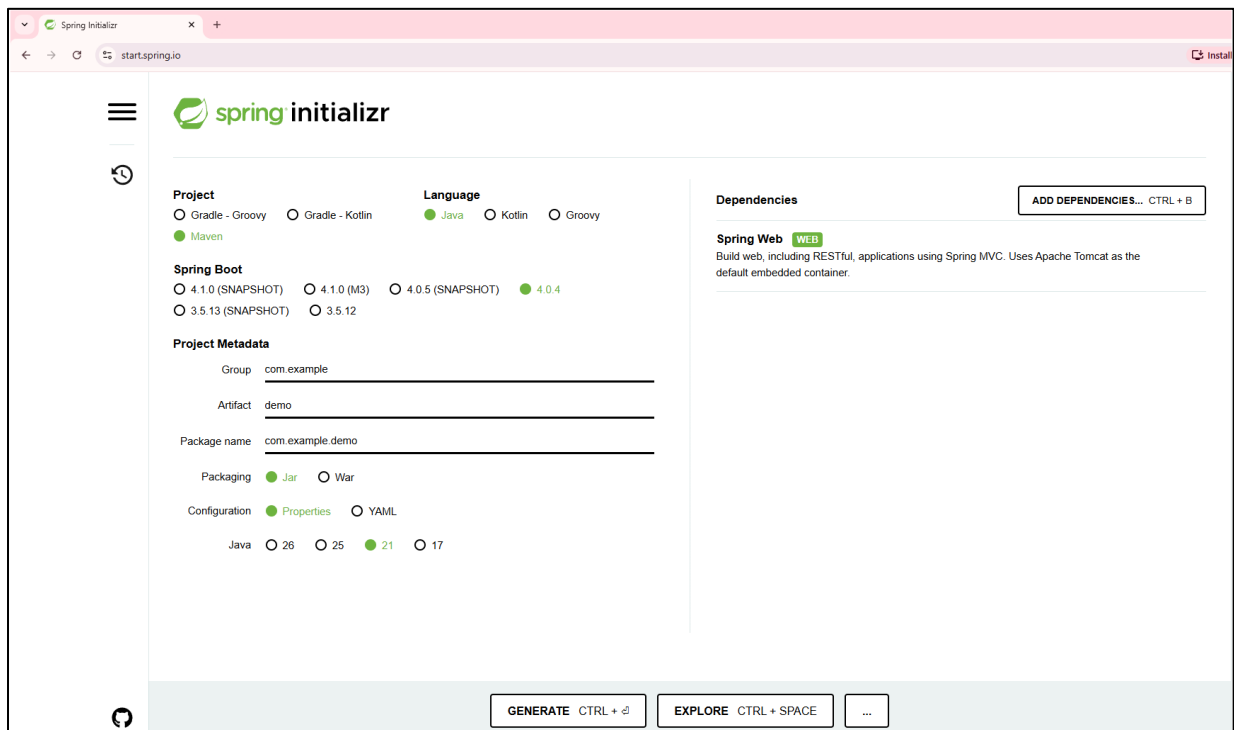


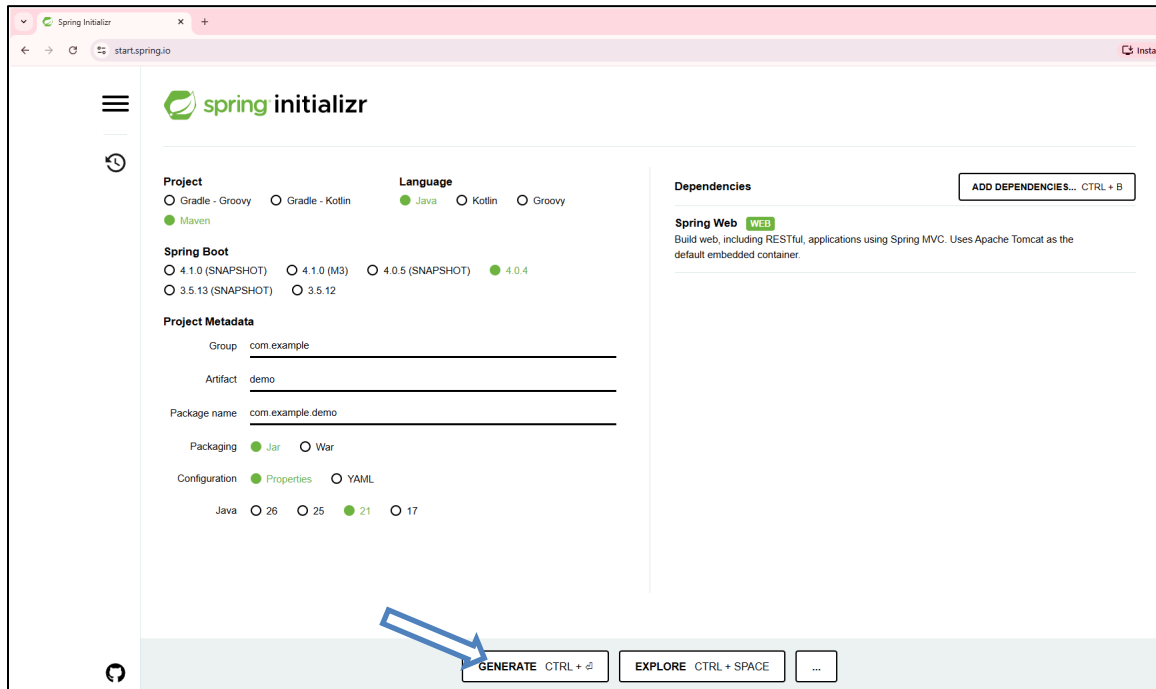
Postman UI



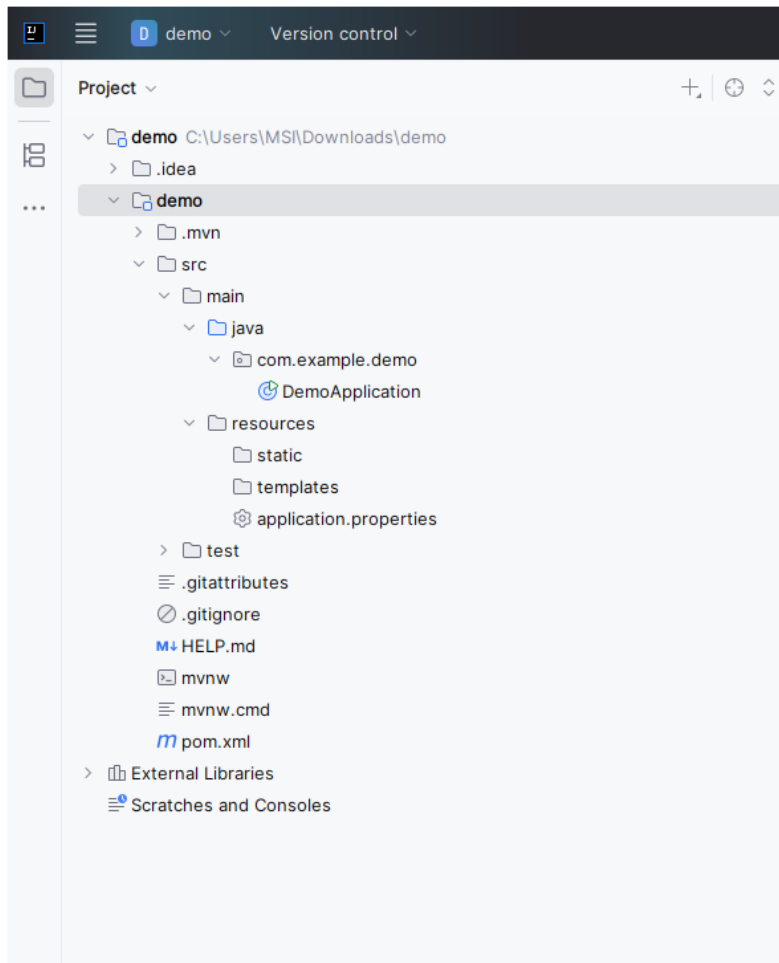
IV. Project Setup

ដើម្បីបង្កើត Project Spring boot បានយើងត្រូវចូលទៅកាន់ Official Website របស់ Spring (start.spring.io) ។





. Default Project Structure



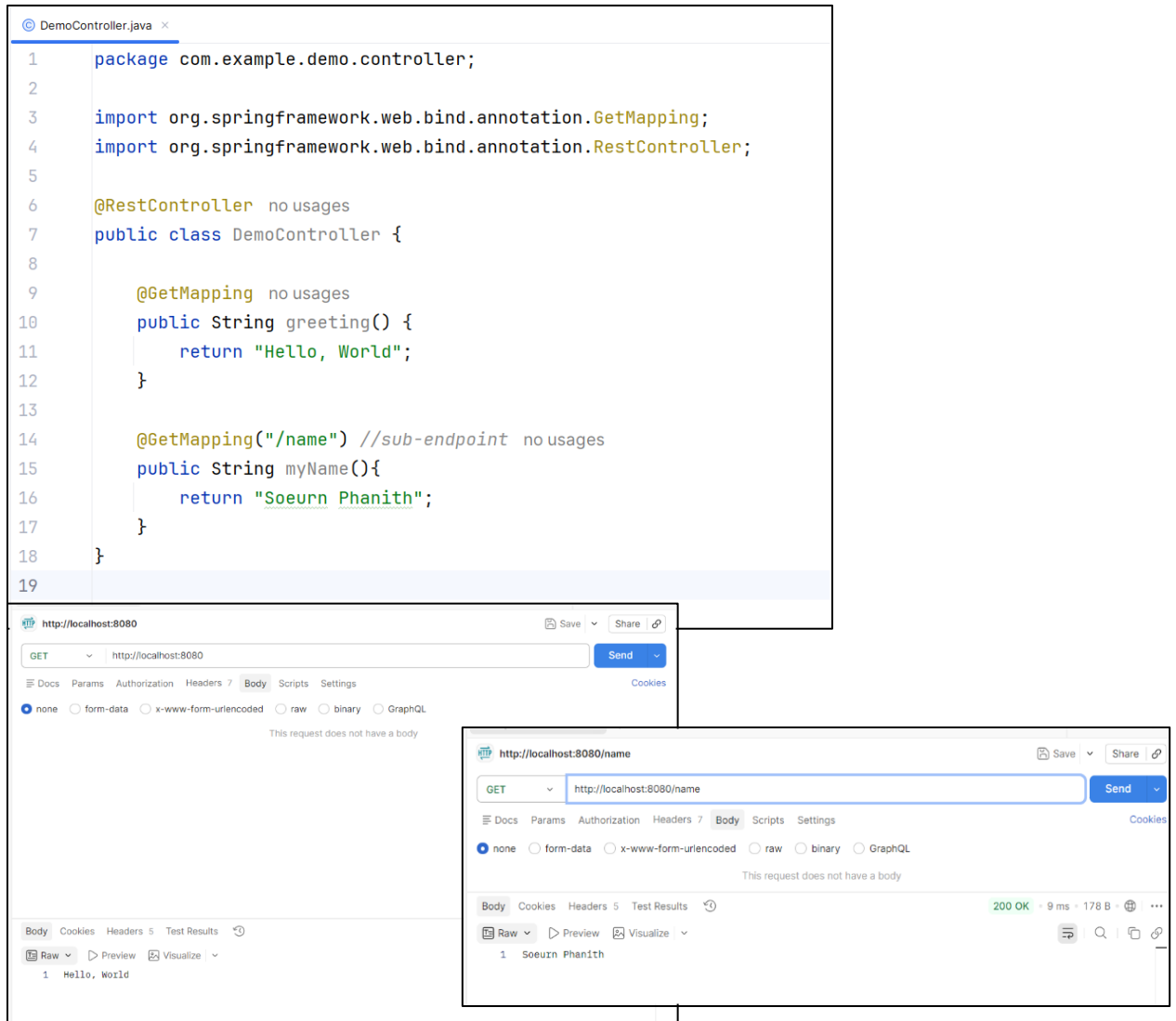
. ណែនាំឲ្យស្គាល់ពី Controller Layer

Controller Layer គឺជា Layerមួយដែលនៅក្នុង Spring Bean ដ៏សំខាន់មួយ ដែលមានតួនាទីនៅក្នុងការដំណើរការ HTTP request ពី Client ហើយបញ្ជូន HTTP response មកកាន់ Client វិញ។

ដោយនៅក្នុង Controller គេចែកជាពីរ ប្រភេទ៖

1. @Controller : សម្រាប់ Spring MVC
2. @RestController : សម្រាប់ RESTful API

Test controller and know about end-point:



The screenshot displays the source code of a Spring MVC controller and its testing results. The code defines a `DemoController` with two endpoints: `greeting()` and `myName()`.

```
1 package com.example.demo.controller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController no usages
7 public class DemoController {
8
9     @GetMapping no usages
10    public String greeting() {
11        return "Hello, World";
12    }
13
14    @GetMapping("/name") //sub-endpoint no usages
15    public String myName(){
16        return "Soeurn Phanith";
17    }
18 }
19
```

Below the code, two REST client test results are shown:

- Test 1:** GET `http://localhost:8080`. The response body is `1 Hello, World`.
- Test 2:** GET `http://localhost:8080/name`. The response body is `1 Soeurn Phanith`. The status is `200 OK` with a response time of `9 ms` and a body size of `178 B`.



Group end-point and dynamic end-point

```
1 package com.example.demo.controller;
2
3 import org.springframework.web.bind.annotation.*;
4
5 @RestController no usages
6 @RequestMapping("/api") //base end-point
7 public class DemoController {
8
9     @GetMapping no usages
10    public String myName(@RequestParam String fullname){return full|name;}
11
12    @GetMapping("/{id}") //dynmaic end-point no usages
13    public String getId(@PathVariable Long id){
14        return "User id = " + id;
15    }
16
17    @GetMapping("/info")//sub-endpoint no usages
18    public String userInformation(
19        @RequestParam String name, @RequestParam String email,
20        @RequestParam String gender, @RequestParam Long age
21    ){
22        return name + "\n" +
23            email + "\n" +
24            gender + "\n" + age;
25    }
26 }
27
```

http://localhost:8080/api/info

GET http://localhost:8080/api/info?name=Soeurn Phanith&email=phanith@gmail.com&gender=male&age=20 Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

name	Soeurn Phanith	
email	phanith@gmail.com	
gender	male	
age	20	
Key	Value	Description

Body Cookies Headers 5 Test Results

Raw Preview Visualize

```
1 Soeurn Phanith
2 phanith@gmail.com
3 male
4 20
```

http://localhost:8080/api/7

GET http://localhost:8080/api/7 Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Document this request... Write with AI

Body Cookies Headers 5 Test Results

Raw Preview Visualize

```
1 User id = 7
```



ដោយនៅក្នុង Controller តែងតែប្រើប្រាស់នៅ HTTP method (annotation) មួយចំនួនដើម្បីធ្វើការជាមួយ ការrequest និង response នៅក្នុងការbuild api ៖

- @GetMapping : ទាញទិន្នន័យ
- @PostMapping : សម្រាប់បញ្ចូលទិន្នន័យ
- @PutMapping : សម្រាប់កែប្រែទិន្នន័យ
- @DeleteMapping : សម្រាប់លុបទិន្នន័យ
- @RequestMapping : base-endpoint

Build crud api to store in list in controller

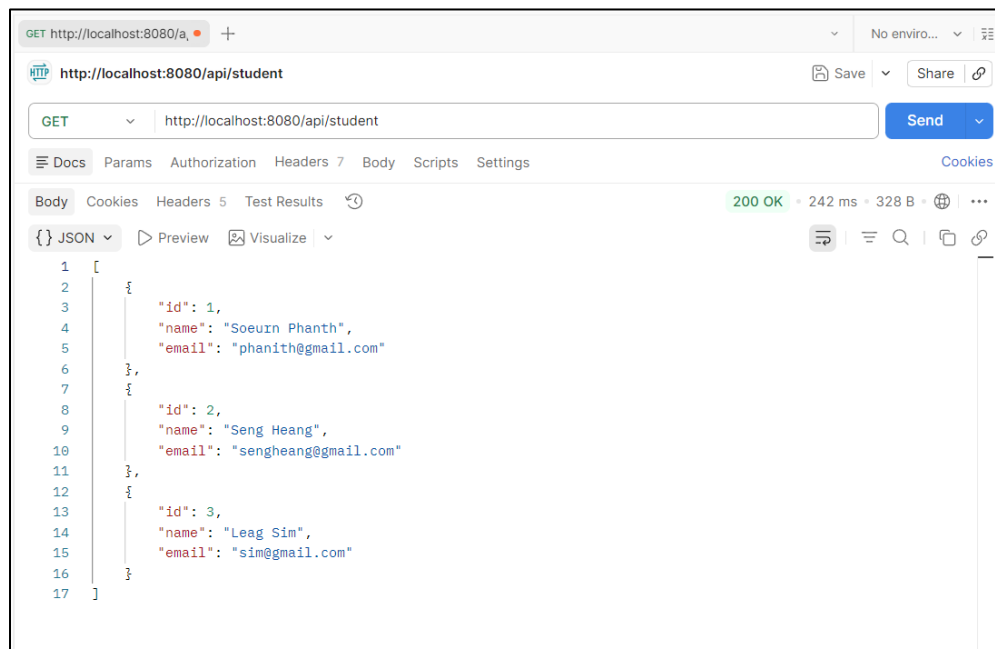
-Create class student

```
1 package com.example.demo.controller;
2
3 public class Student { no usages
4     private Long id; 4 usages
5     private String name; 4 usages
6     private String email; 4 usages
7
8     public Student(Long id, String name, String email) { no usages
9         this.id = id;
10        this.name = name;
11        this.email = email;
12    }
13    public Student(){ no usages
14        this.id = Long.valueOf(1: 0);
15        this.name = "unknown";
16        this.email = "unkown";
17    }
18
19    public Long getId() {return id;} no usages
20    public void setId(Long id) {this.id = id;} no usages
21    public String getName() {return name;} no usages
22    public void setName(String name) {this.name = name;} no usages
23    public String getEmail() {return email;} no usages
24    public void setEmail(String email) {this.email = email;} no usages
25 }
26
27
```



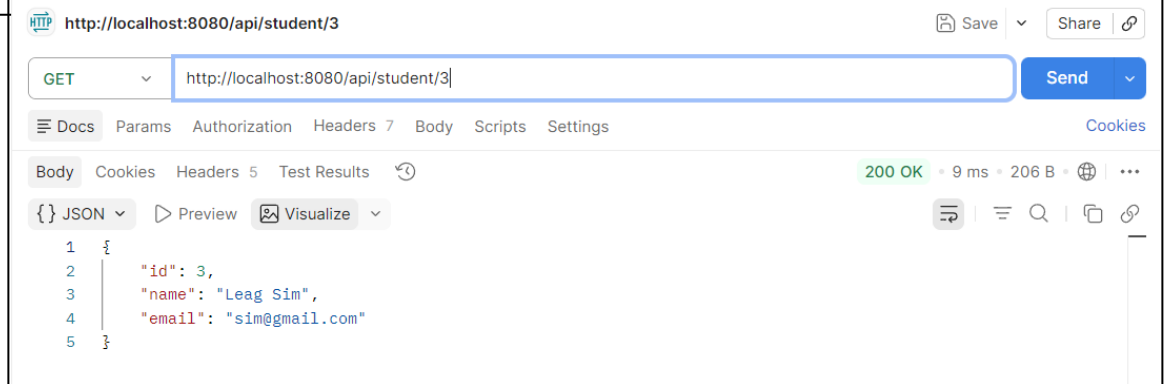
- Show all data of student

```
1 package com.example.demo.controller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RequestMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 import java.util.ArrayList;
8 import java.util.List;
9
10 @RestController no usages
11 @RequestMapping("/api/student")
12 public class StudentController {
13
14     List<Student> allStudent = new ArrayList<>() 1 usage
15         List.of(
16             new Student(id: 1L, name: "Soeurn Phanth", email: "phanith@gmail.com"),
17             new Student(id: 2L, name: "Seng Heang", email: "sengheang@gmail.com"),
18             new Student(id: 3L, name: "Leag Sim", email: "sim@gmail.com")
19         )
20 );
21
22 @GetMapping no usages
23 public List<Student> getAllStudent(){
24     return allStudent;
25 }
26
27 }
28
```



- Get one Student

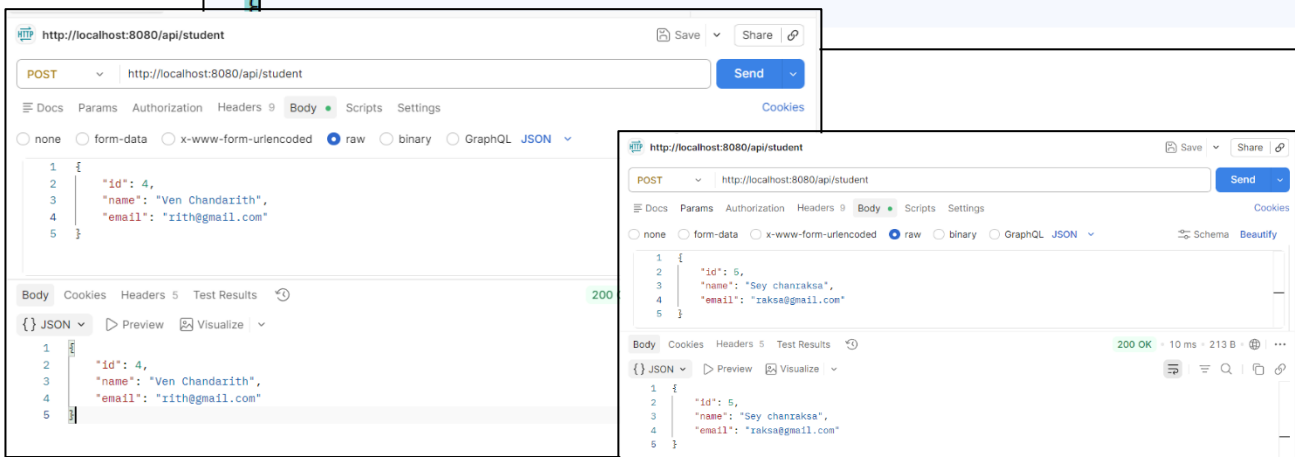
```
@GetMapping("/{id}")
public Student getOneStudent(@PathVariable Long id) {
    return allStudent.stream() Stream<Student>
        .filter( Student s -> s.getId().equals(id))
        .findAny() Optional<Student>
        .orElseThrow(() -> new RuntimeException("student not found"));
}
```



- Create Student

```
@PostMapping no usages
public Student createNewStudent(@RequestBody Student student){
    Student s = new Student(
        student.getId(),
        student.getName(),
        student.getEmail()
    );
    allStudent.add(s);

    return s;
}
```



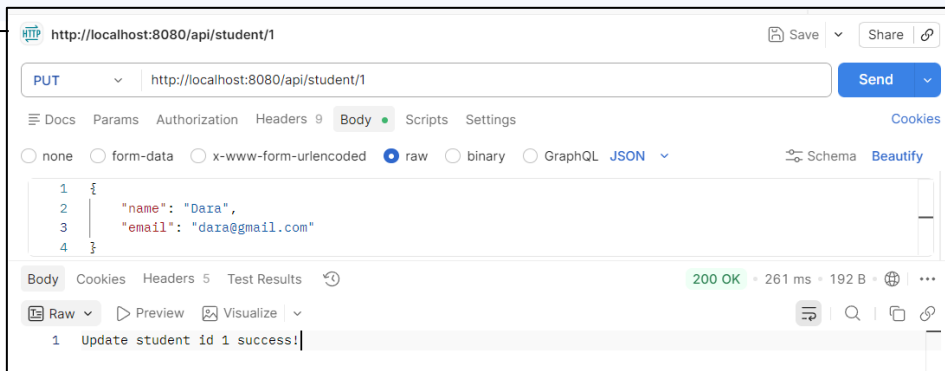


- Update student by id

```
@PutMapping("/{id}")
public String updateStudent(@PathVariable Long id, @RequestBody Student student){
    Student find = allStudent.stream() Stream<Student>
        .filter( Student s->s.getId().equals(id))
        .findAny() Optional<Student>
        .orElseThrow(()-> new RuntimeException("student not found"));

    //update
    find.setName(student.getName());
    find.setEmail(student.getEmail());

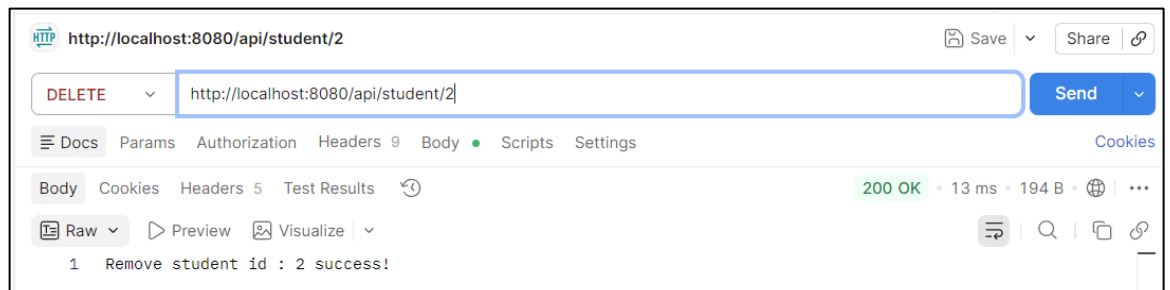
    return "Update student id "+ id + " success!";
}
```



- Delete Student

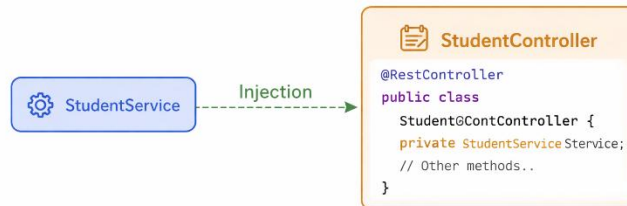
```
@DeleteMapping("/{id}") no usages
public String deleteStudent(@PathVariable Long id){
    Student studentForRemove = allStudent.stream() Stream<Student>
        .filter( Student s->s.getId().equals(id))
        .findAny() Optional<Student>
        .orElseThrow(()-> new RuntimeException("student not found"));

    allStudent.remove(studentForRemove);
    return "Remove student id : " + id + " success!";
}
```



. ណែនាំឲ្យស្គាល់ពី Service Layer

Service layer គឺជា layer ដែលរៀបចំ business logic មួយដែលវា manage ឬរាប់រងលើការរៀបចំទិន្នន័យ និងដំណើរការទិន្នន័យ។ ដោយរាល់កូដឬការងារទាំងអស់ត្រូវសរសេរ និងរៀបចំក្នុង Service ទើបយកទៅប្រើប្រាស់នៅ Controller តាមក្រោយ ដើម្បីឲ្យមាន Structure ល្អនៃ Project។



- GetAllStudent and GetOneStudent in Service

```
© DemoController.java  © StudentService.java  ×  © StudentController.java  © Student.java  © DemoApplication.java
4  import org.springframework.stereotype.Service;
5  import org.springframework.web.bind.annotation.*;
6
7  import java.util.ArrayList;
8  import java.util.List;
9
10 @Service no usages
11 public class StudentService {
12
13     List<Student> allStudent = new ArrayList<>( 6 usages
14         List.of(
15             new Student(id: 1L, name: "Soeurn Phan", email: "phanith@gmail.com"),
16             new Student(id: 2L, name: "Seng Heang", email: "sengheang@gmail.com"),
17             new Student(id: 3L, name: "Leag Sim", email: "sim@gmail.com")
18         )
19     );
20
21     public List<Student> getAllStudent(){ no usages
22         return allStudent;
23     }
24
25     public Student getOneStudent(Long id) { no usages
26         return allStudent.stream() Stream<Student>
27             .filter( Student s -> s.getId().equals(id))
28             .findAny() Optional<Student>
29             .orElseThrow(() -> new RuntimeException("student not found"));
30     }
31 }
```

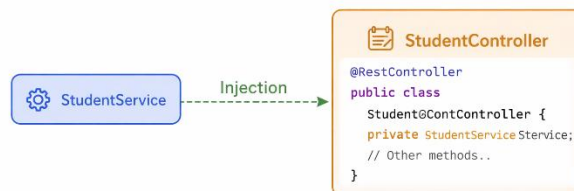
- Create Student , Update, Delete Student

```

11 public class StudentService {
30     }
31
32     @ public Student createNewStudent(Student student){ no usages
33         Student s = new Student(
34             student.getId(),
35             student.getName(),
36             student.getEmail()
37         );
38         allStudent.add(s);
39         return s;
40     }
41
42     @ public String updateStudent(Long id, Student student){ no usages
43         Student find = allStudent.stream().filter( Student s->s.getId().equals(id))
44             .findAny().orElseThrow(()-> new RuntimeException("student not found"));
45         find.setName(student.getName());
46         find.setEmail(student.getEmail());
47         return "Update student id "+ id + " success!";
48     }
49
50     public String deleteStudent(Long id){ no usages
51         Student studentForRemove = allStudent.stream().filter( Student s->s.getId().equals(id))
52             .findAny().orElseThrow(()-> new RuntimeException("student not found"));
53         allStudent.remove(studentForRemove);
54         return "Remove student id : " + id + " success!";
55     }
56

```

បន្ទាប់ពីរៀបចំការងារ ឬ logic code នៅក្នុង service layer រួចរាល់ហើយ ត្រូវ Inject រាយកទៅដាក់ក្នុង controller layer ដើម្បីឲ្យរាងដំណើរការ រាល់request នីមួយៗពីClient តាមរយៈ DI (Dependency Injection)។



- Inject data from Service put in Controller(DI)

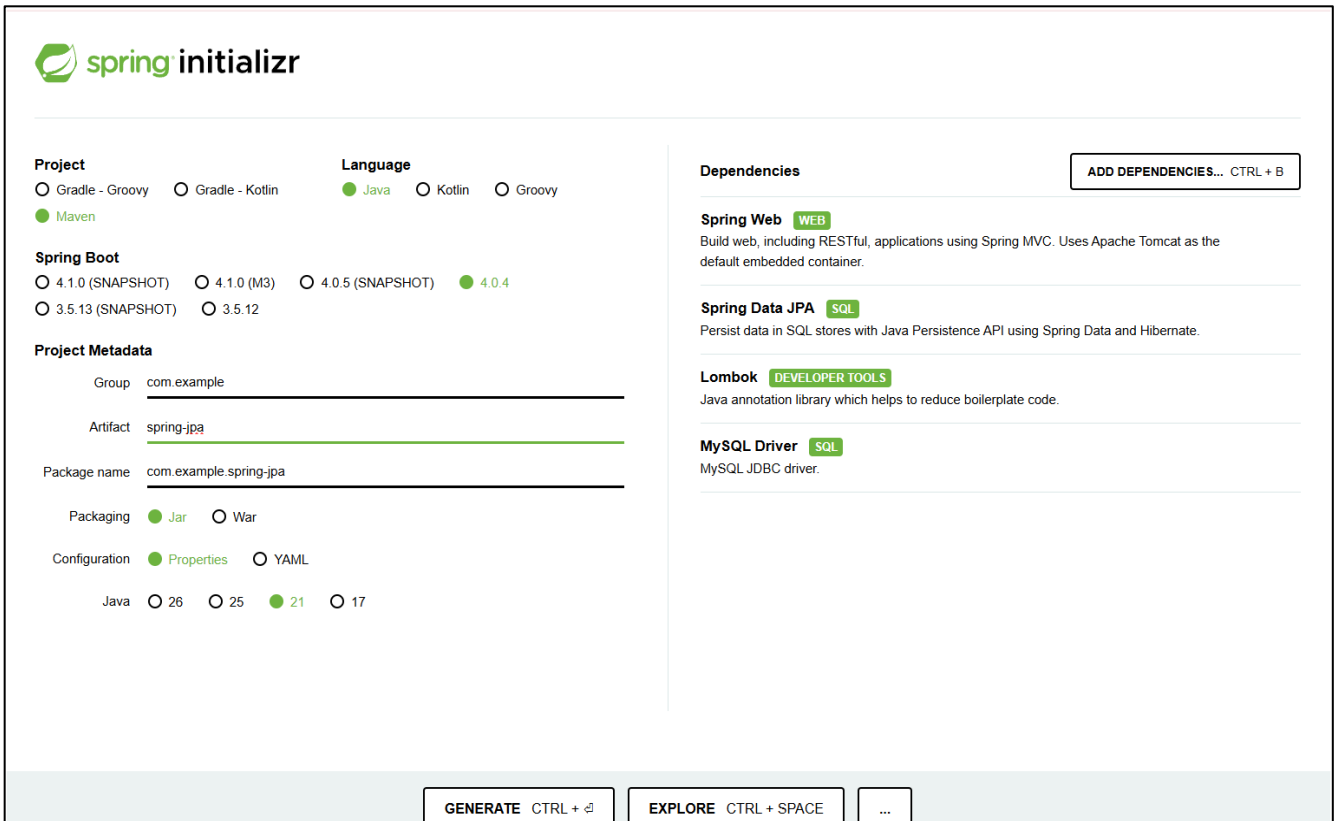
```
1 package com.example.demo.controller;|
2 import com.example.demo.service.StudentService;
3 import org.springframework.web.bind.annotation.*;
4 import java.util.List;
5
6 @RestController no usages
7 @RequestMapping("/api/student")
8 public class StudentController {
9
10     private final StudentService service; 5 usages
11     public StudentController(StudentService service) { no usages
12         this.service = service;
13     }
14
15     @PostMapping no usages
16     public Student createStudent(@RequestBody Student student){
17         return service.createNewStudent(student);
18     }
19
20     @GetMapping no usages
21     public List<Student> showAllStudent(){
22         return service.getAllStudent();
23     }
24
25     @PutMapping("/{id}") no usages
26     public String updateStudent(@PathVariable Long id, @RequestBody Student s){
27         return service.updateStudent(id,s);
28     }
29
30     @DeleteMapping("/{id}") no usages
31     public String deleteStudent(@PathVariable Long id){
32         return service.deleteStudent(id);
33     }
34 }
35
```

បន្ទាប់ពី Inject Bussiness Logic ពី Service layer មកដាក់ក្នុង Controller រួចហើយវានឹងដំណើរការ លើការងារនោះធម្មតាហើយ។ ប៉ុន្តែកូដរបស់យើងវា clean ជាងមុន។

V. Spring Data Jpa (database connection)

Spring Data JPA គឺជា module មួយក្នុង Spring Framework ដែលជួយសម្រួលការធ្វើការជាមួយ database ដោយប្រើ JPA (Java Persistence API) និងកាត់បន្ថយការសរសេរ SQL code។ ដោយប្រើសម្រាប់ map data ពី object ទៅជា table ក្នុង database ព្រមទាំង control លើការធ្វើ ឬសរសេរ query ក្នុង database ផងដែរ។

. set up project with spring data jpa



The image shows the Spring Initializr web interface for configuring a new project. The configuration is as follows:

- Project:** ☐ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ Java, ☐ Kotlin, ☐ Groovy
- Language:** ☒ Maven
- Spring Boot:** ☐ 4.1.0 (SNAPSHOT), ☐ 4.1.0 (M3), ☐ 4.0.5 (SNAPSHOT), ☒ 4.0.4, ☐ 3.5.13 (SNAPSHOT), ☐ 3.5.12
- Project Metadata:**
 - Group: com.example
 - Artifact: spring-jpa
 - Package name: com.example.spring-jpa
 - Packaging: ☒ Jar, ☐ War
 - Configuration: ☒ Properties, ☐ YAML
 - Java: ☐ 26, ☐ 25, ☒ 21, ☐ 17
- Dependencies:**
 - Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
 - Spring Data JPA** (SQL): Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
 - Lombok** (DEVELOPER TOOLS): Java annotation library which helps to reduce boilerplate code.
 - MySQL Driver** (SQL): MySQL JDBC driver.

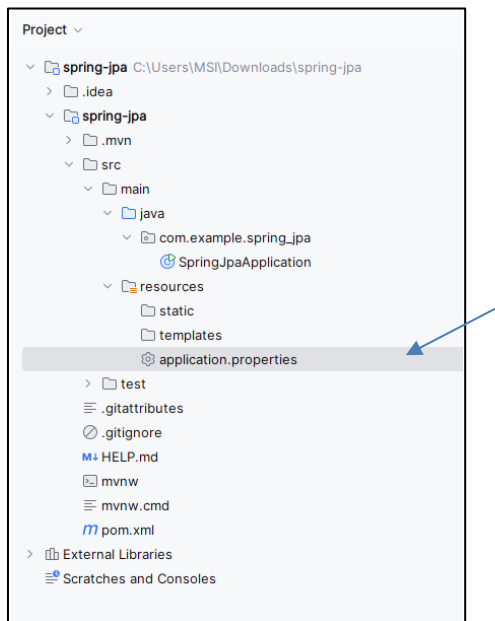
Buttons at the bottom:

ដោយ ៖

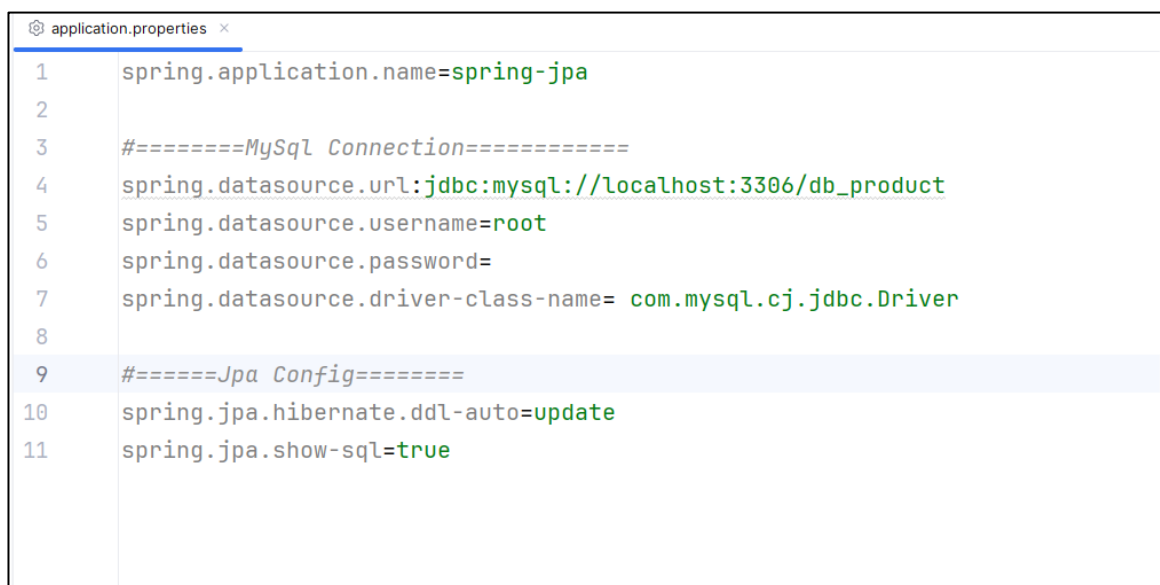
- . Spring Data JPA : framework សម្រាប់ធ្វើការជាមួយ database។
- . Lombok : framework or library សម្រាប់ជំនួសការប្រើ setter & getter....
- . MySQL Driver : ជា driver សម្រាប់ connection to MySQL

. ណែនាំឲ្យស្គាល់ពី application.propoties or yml

application.propoties or application.yml ទាំងពីរនេះគឺជា configuration file ក្នុង Spring Bootដែលប្រើសម្រាប់កំណត់ settings ដូចជា: Port , Database ,Logging ,Custom config.....។



1. Configuration តាម application.propoties



2. Configuration តាម application.yml

```
application.yml x
1  spring:
2    application:
3      name: spring-jpa
4
5    datasource:
6      url: jdbc:mysql://localhost:3306/db_product
7      username: root
8      password:
9      driver-class-name: com.mysql.cj.jdbc.Driver
10
11   jpa:
12     show-sql: true
13     hibernate:
14       ddl-auto: update
15
16   server:
17     port: 8080
```

. ណែនាំឲ្យស្គាល់ពី Model / Entity

Entity/Model គឺជា class ដែលដំណាងឲ្យ Table នៅក្នុង database ដោយរក hibernate JPA ហើយធ្វើឲ្យ Object នៃ Entity នោះទៅជា Table។

.Annotation ដែលប្រើនៅក្នុង entity ៖

- @Entity : ប្រាប់នៅកាន់ Spring ថាវាជា Bean និងជា class នៃ table
- @Table : custom name of table
- @Id : ឲ្យ Id ជា Primary Key
- @GeneratedValue(.....) : Id auto-increment
- @Column : custom column in table

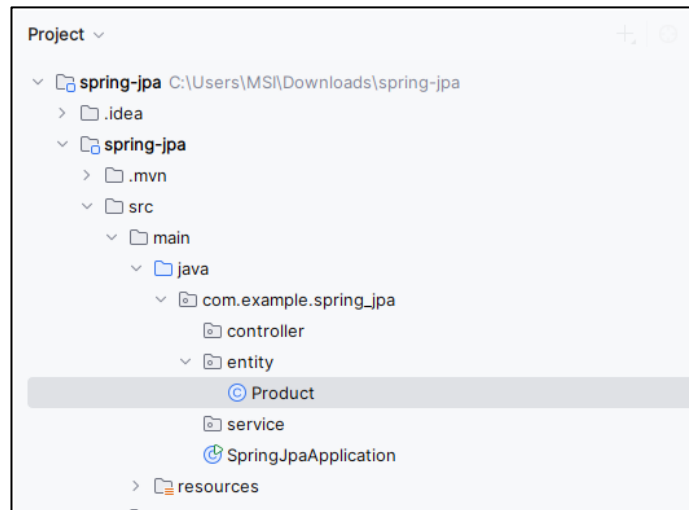
Example :

@Column(nullable = true)

private String name;



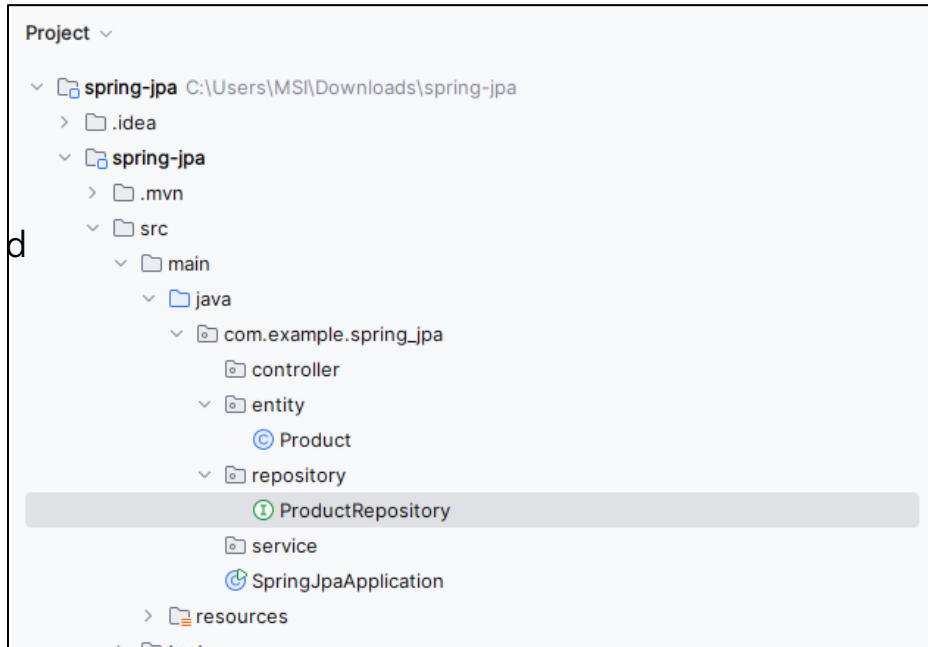
- Create class product to make to table



```
1 package com.example.spring_jpa.entity;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import java.math.BigDecimal;
6
7 @AllArgsConstructor no usages
8 @NoArgsConstructor
9 @Data
10 @Builder
11 @Entity
12 @Table(name = "tbl_product")
13 public class Product {
14
15     @Id no usages
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     @Column(name = "product_name") no usages
20     private String name;
21
22     private BigDecimal price; no usages
23
24     private String description; no usages
25
26
27 }
28
```

. ណែនាំឲ្យស្គាល់ពី Repository Layer

Repository (DAO) គឺជា Interface ដែលប្រើសម្រាប់ធ្វើការជាមួយ Database (CRUD operations) ពេលគឺវាមាន តួនាទីនៅក្នុងការផ្តល់ទៅលើ query ដែលត្រូវប្រើប្រាស់ដោយមិនចាំបាច់សរសេរនៅក្នុង database ផ្ទាល់ទេ។

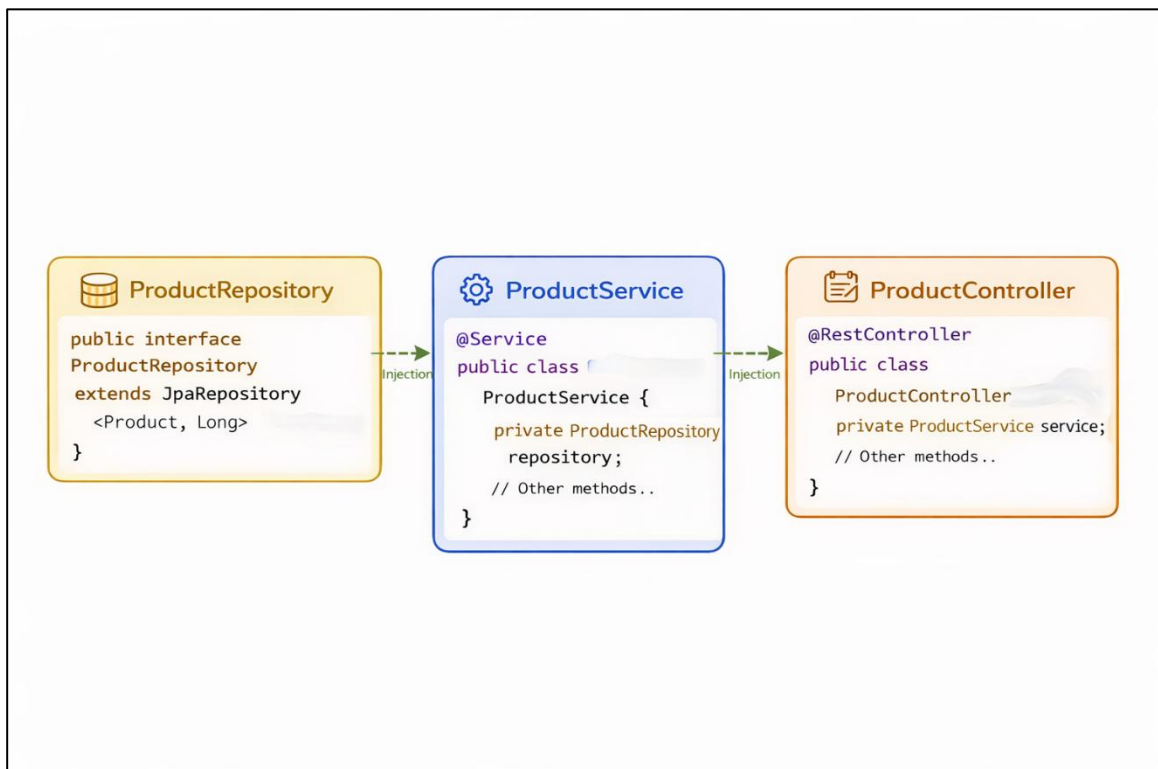


ដោយ Repository វា inheritance (extends) ពី JpaRepository ដូច្នេះយើងអាចប្រើប្រាស់នូវ Query ផ្សេងៗដែលផ្ទុកក្នុង method នៃ JpaRepository នោះបានដោយមិនចាំបាច់សរសេរ Raw Query or Query builderឡើយ។

Build Api to store data in database using Spring data Jpa

- ធ្វើការរៀបចំការងាររបស់code ឬ business logic នៃcode Service layer ដោយធ្វើការ inject ProductRepository មកកាន់ server ដើម្បីប្រើប្រាស់នូវ Query បាន។

```
1 package com.example.spring_jpa.service;
2
3 import com.example.spring_jpa.repository.ProductRepository;
4 import org.springframework.stereotype.Service;
5
6 @Service no usages
7 public class ProductService {
8
9     //Dependencies Injection
10    private final ProductRepository productRepo; 1 usage
11    public ProductService(ProductRepository productRepo) { no usages
12        this.productRepo = productRepo;
13    }
14 }
15
```

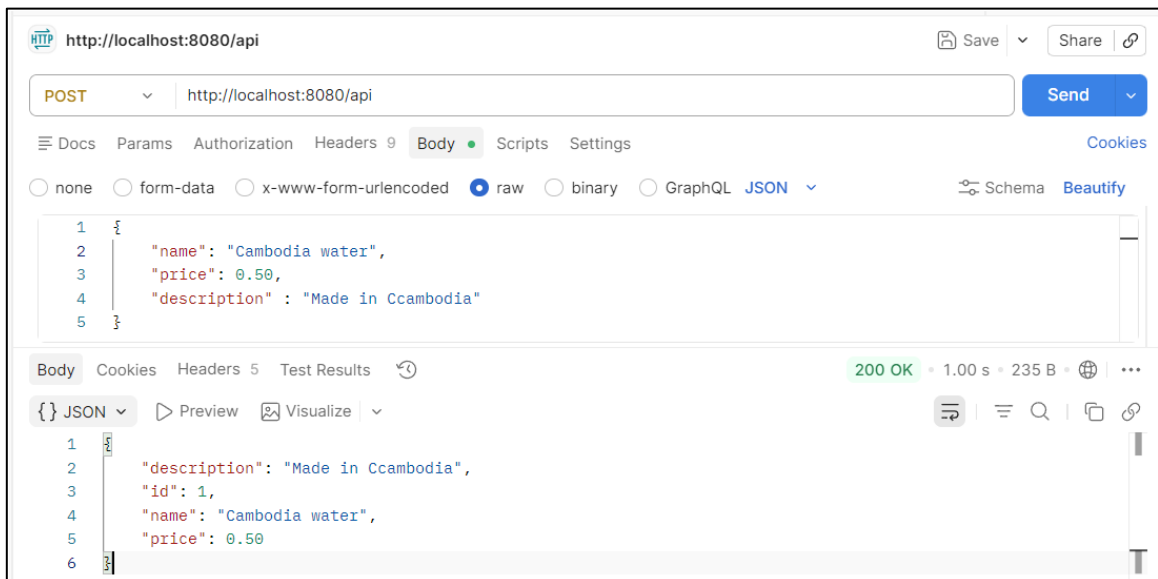


- Create product in service

```

16 //method create product
17 @public Product addProduct(Product product){ no usages
18     //check exists product by name
19     boolean existsProduct = productRepo.existsByName(product.getName());
20     if(existsProduct){
21         throw new RuntimeException("product name is already exists");
22     }
23
24     Product p = new Product(
25         product.getName(),
26         product.getPrice(),
27         product.getDescription()
28     );
29
30     //save to database
31     Product saved = productRepo.save(p);
32     return saved;
33 }
34 }

```



Extra options

↩ T ↪

▼

id

product_name

price

description

☐	 Edit	 Copy	 Delete	1	Cambodia water	0.50	Made in Ccambodia
--------------------------	--	--	--	---	----------------	------	-------------------

⬆

☐ Check all

With selected:

 Edit

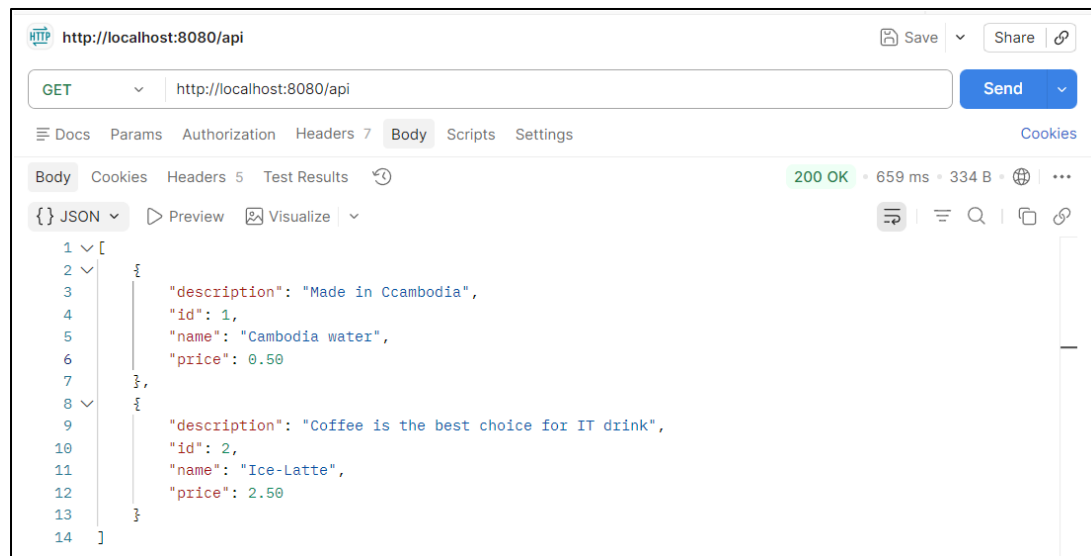
 Copy

 Delete

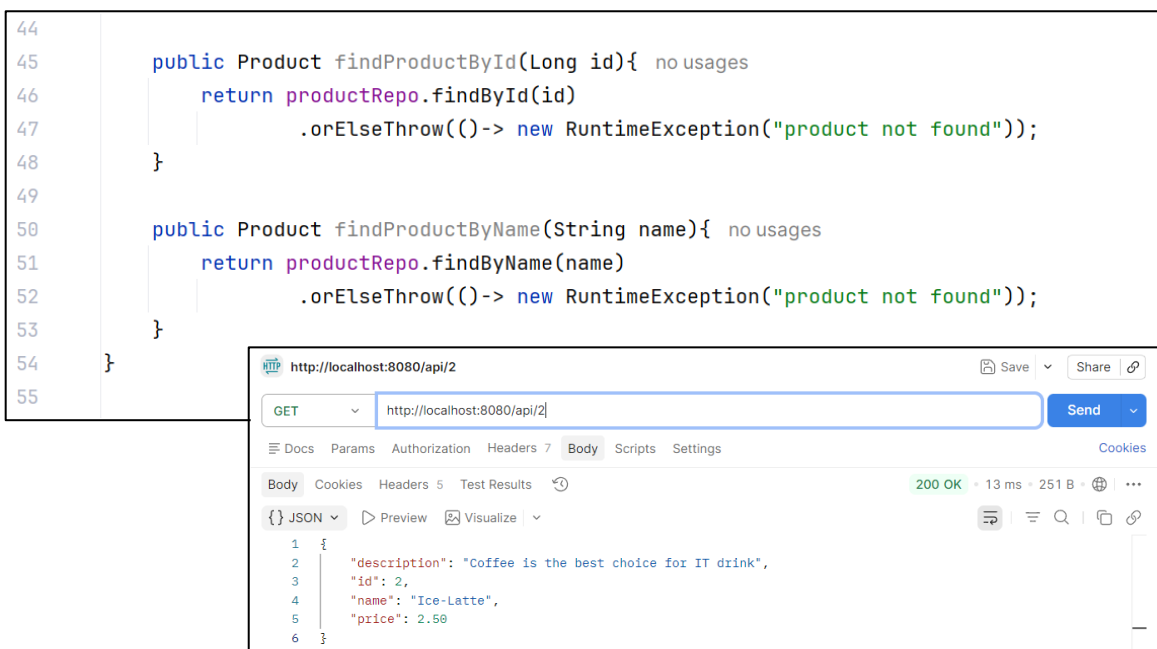
 Export

- Create method to show all product

```
36
37     public List<Product> showAllProduct(){ no usages
38         List<Product> allProduct = productRepo.findAll();
39         if(allProduct.isEmpty()){
40             throw new RuntimeException("No product record");
41         }
42         return allProduct;
43     }
44 }
```



- Show One Product By Id or Name





- Update product by id

```
54
55 @
56 public Product updateProduct(Long id, Product product){ no usages
57     Product existing = productRepo.findById(id)
58         .orElseThrow(() -> new RuntimeException("product not found"));
59
60     //check duplicate name
61     if(productRepo.existsByName(product.getName())
62         && !existing.getName().equals(product.getName())){
63         throw new RuntimeException("product name already exists");
64     }
65     existing.setName(product.getName());
66     existing.setPrice(product.getPrice());
67     existing.setDescription(product.getDescription());
68     return productRepo.save(existing);
69 }
70
71
```

HTTP client interface showing a PUT request to `http://localhost:8080/api/1` with a JSON body:

```
{
  "name": "Milk",
  "price": 0.75,
  "description": "Milk is a good drink for kid and good healthy"
}
```

The response is `200 OK` with a status bar showing `1.24 s` and `253 B`. The response body is displayed in JSON format:

```
{
  "description": "Milk is a good drink for kid and good healthy",
  "id": 1,
  "name": "Milk",
  "price": 0.75
}
```

Extra options

↩

⌵

→

id

product_name

price

description

☐	Edit	Copy	Delete	1	Milk	0.75	Milk is a good drink for kid and good healthy
☐	Edit	Copy	Delete	2	Ice-Latte	2.50	Coffee is the best choice for IT drink

⬆

☐ Check all

With selected:

Edit

Copy

Delete

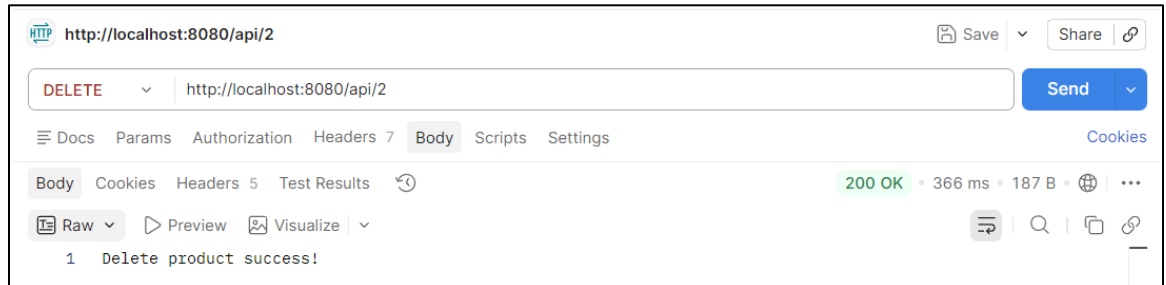
Export

- Delete Product

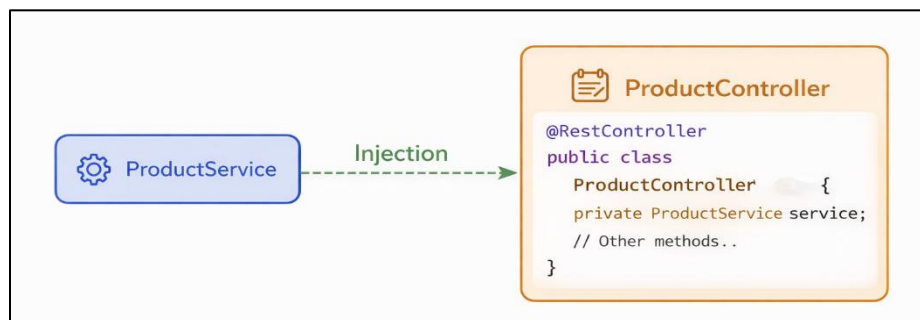
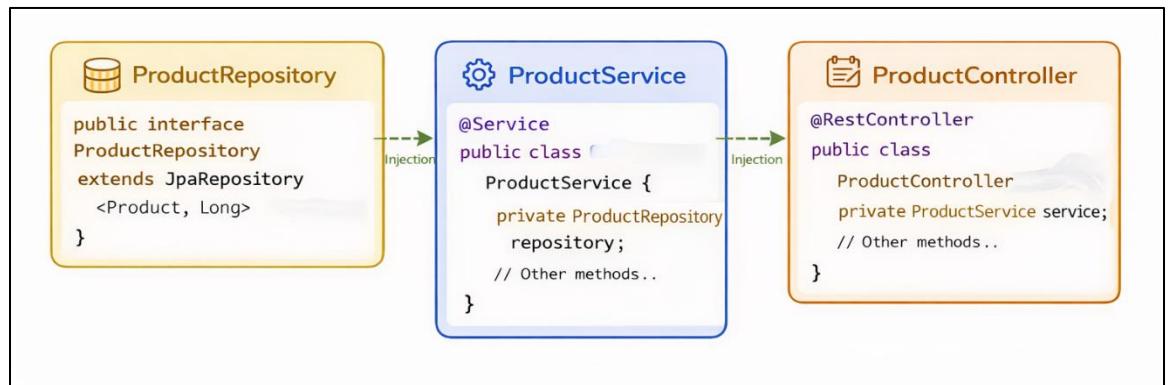
```

70
71     public void deleteProduct(Long id){ no usages
72
73         Product existing = productRepo.findById(id)
74             .orElseThrow(() -> new RuntimeException("product not found"));
75
76         productRepo.delete(existing);
77     }
78 }

```



- បន្ទាប់ពីបានរៀបចំការងារនៅក្នុង Service រួចរាល់ហើយ ត្រូវ ធ្វើការ injection method ទាំងនោះយកមកដាក់ដំណើការនៅក្នុង Controller ដូចរាល់ដង។





- Inject into Controller

```
ProductController.java x
1 package com.example.spring_jpa.controller;|
2
3 import com.example.spring_jpa.entity.Product;
4 import com.example.spring_jpa.service.ProductService;
5 import org.springframework.web.bind.annotation.*;
6 import java.util.List;
7
8 @RestController no usages
9 @RequestMapping("/api")
10 public class ProductController {
11
12     private final ProductService productService; 7 usages
13     public ProductController(ProductService productService) { no usages
14         this.productService = productService;
15     }
16
17     @PostMapping no usages
18     public Product addProduct(@RequestBody Product product){
19         return productService.addProduct(product);
20     }
21
22     @GetMapping no usages
23     public List<Product> allProduct(){
24         return productService.showAllProduct();
25     }
26
27     @GetMapping("/{id}") no usages
28     public Product findProductById(@PathVariable Long id){
29         return productService.findProductById(id);
30     }
31
32     @GetMapping("/find-name/{name}") no usages
33     public Product findProductByName(@PathVariable String name){
34         return productService.findProductByName(name);
35     }
36
37     @PutMapping("/{id}") no usages
38     public Product updateProduct(@PathVariable Long id, @RequestBody Product p){
39         return productService.updateProduct(id, p);
40     }
41
42     @DeleteMapping("/{id}") no usages
43     public String deleteProductById(@PathVariable Long id){
44         productService.deleteProduct(id);
45
46         return "Delete product success!";
47     }
48 }
49
```



លំហាត់អនុវត្តន៍

ចូរបង្កើតនូវ Api មួយទាក់ទងទៅនឹង Teacher ដែលមាន field or columnដូចជា Id, name, gender, skill, salary, schoolName, createdAt, updatedAt ដោយ follow ទៅតាម requirement :

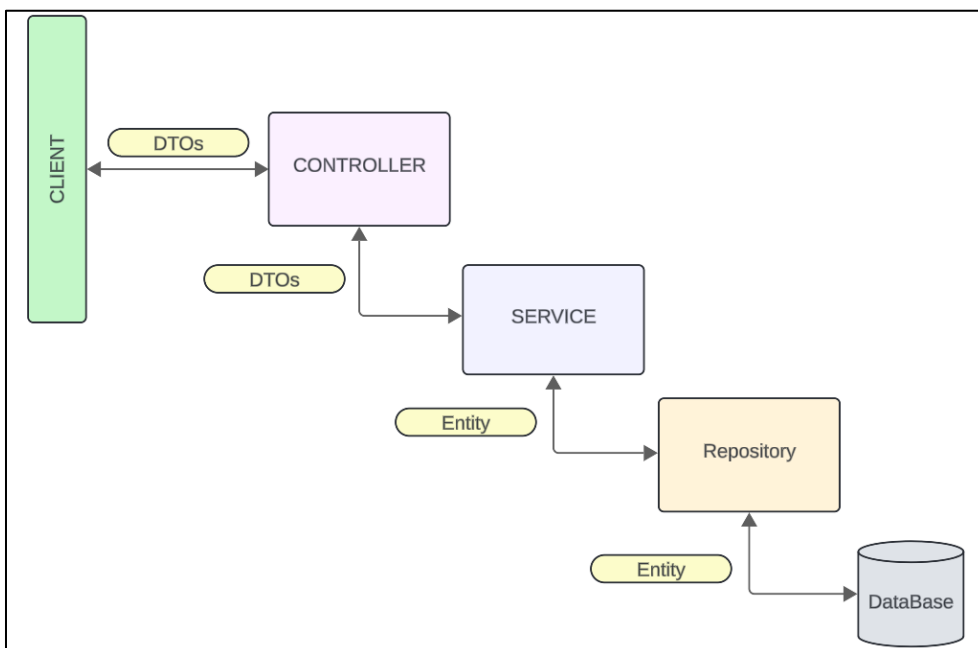
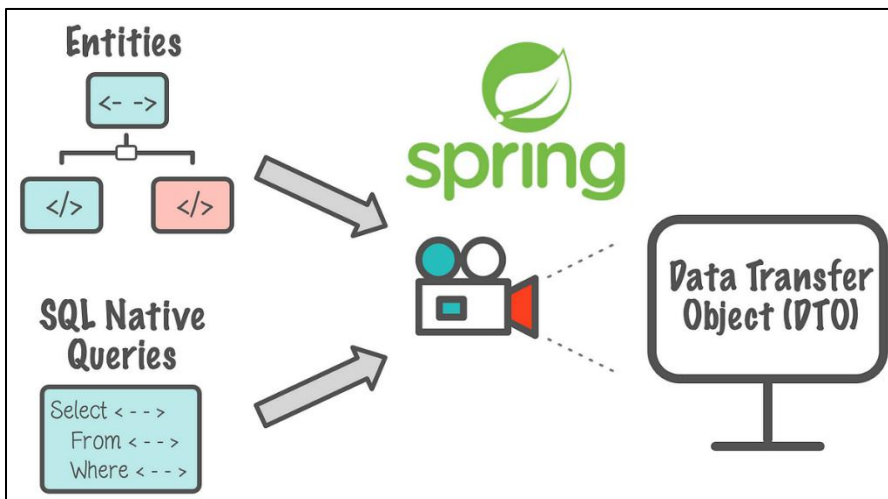
1. To do Crud Operation
2. Validate all of field what you know when input
3. Check exists Teacher name
4. Find Teacher by id
5. Find teacher by salary
6. Delete Teacher by name

GET	/pet/{petId}	Find pet by ID
PUT	/pet	Update an existing pet
DELETE	/pet/{petId}	Deletes a pet
POST	/pet/{petId}/uploadImage	uploads an image

. ណែនាំឲ្យស្គាល់ពី DTOs Layer

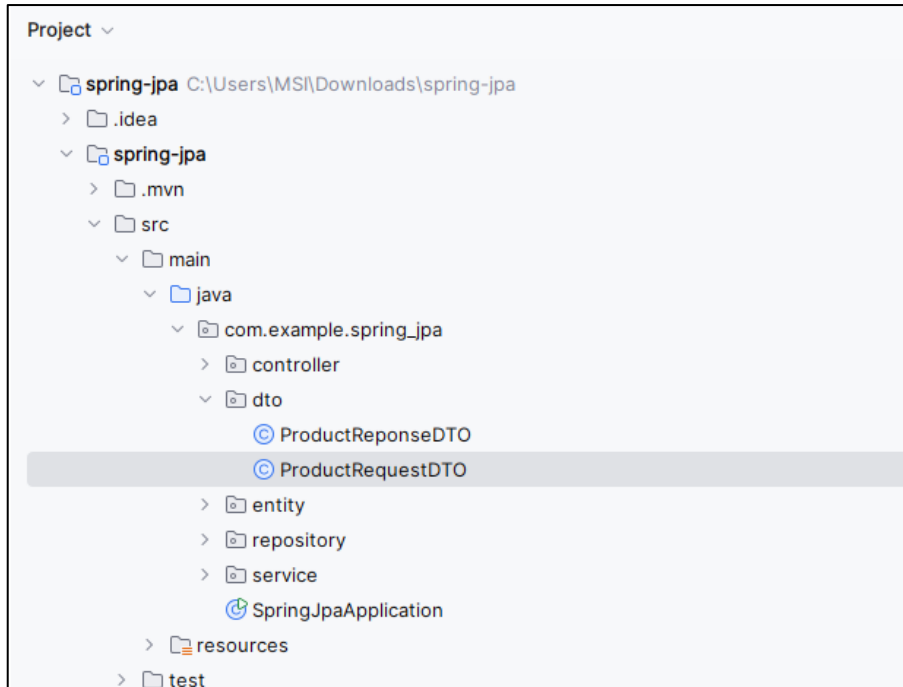
DTOs (Data Transfer Object) គឺជា Layer មួយដែលប្រើសម្រាប់ផ្លាស់ប្តូរទិន្នន័យរវាង Entity/Model ជាមួយនឹង Client ហើយវាមិនឲ្យ Client ធ្វើការដោយផ្ទាល់ជាមួយ database ទេ ។

DTOs គេចែកជា ២ ប្រភេទ គឺ Request DTO និង Response DTO។ ដោយ Request DTOជា អ្នកចាំទទួល Request ពី Client Side មក។ រី Response DTO វិញគឺជាអ្នកដែល Response data ទៅកាន់ Client វិញ ប៉ុន្តែផ្តល់តែData ណាដែលចាំបាច់ប៉ុណ្ណោះ ។





- Create Request DTOs





- Create Response DTOs

```
© ProductController.java  © ProductRequestDTO.java  © ProductResponseDTO.java x
1  package com.example.spring_jpa.dto;
2
3  import lombok.Getter;
4  import lombok.Setter;
5  import java.math.BigDecimal;
6
7  @Setter no usages
8  @Getter
9  public class ProductReponseDTO {
10
11     private Long id; no usages
12
13     private String name; no usages
14
15     private BigDecimal price; no usages
16
17 }
18
```

VI. File Upload or Image Upload

Image Upload API គឺជា RESTful API endpoint ដែលអនុញ្ញាតឱ្យ client (Web Browser, Mobile App, Postman...) ផ្ញើ file image ទៅ server។

- Configuration on application.properties or yml

```
15
16     # Upload folder and limit file
17     file.upload-dir=uploads
18     spring.servlet.multipart.max-file-size=30MB
19     spring.servlet.multipart.max-request-size=30MB
```

- Create folder upload in root directory

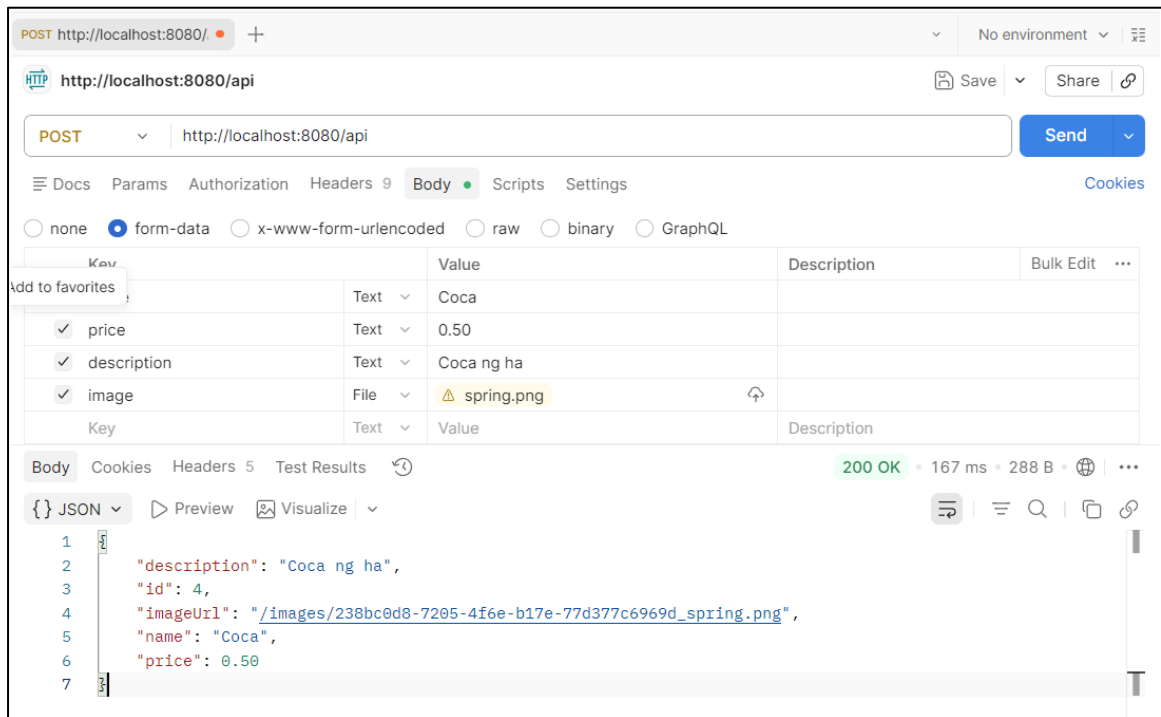
```
ProductResponseDTO.java  SpringJpaApplication.java  Product.java  ProductService.java  ImagePathConfig.java x  application.properties
1  package com.example.spring_jpa.config;
2
3  import org.springframework.beans.factory.annotation.Value;
4  import org.springframework.context.annotation.Bean;
5  import org.springframework.context.annotation.Configuration;
6
7  import java.io.File;
8  import java.nio.file.Files;
9  import java.nio.file.Path;
10 import java.nio.file.Paths;
11
12 @Configuration no usages
13 public class ImagePathConfig {
14
15     @Value("${file.upload-dir}") 1 usage
16     private String uploadDir;
17
18     @Bean no usages
19     public Path uploadsPath() throws Exception {
20         Path path = Paths.get(System.getProperty("user.dir")).resolve(uploadDir);
21         if (!Files.exists(path)) {
22             Files.createDirectories(path);
23         }
24         return path;
25     }
26
27 }
```




- Manage business logic in Service layer for upload

```
ProductResponseDTO.java  SpringJpaApplication.java  Product.java  ProductService.java  ImagePathConfig.java  application.properties
1  package com.example.spring_jpa.service;
2
3  import com.example.spring_jpa.entity.Product;
4  import com.example.spring_jpa.repository.ProductRepository;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.stereotype.Service;
7  import org.springframework.web.multipart.MultipartFile;
8  import java.io.IOException;
9  import java.nio.file.Path;
10 import java.util.List;
11 import java.util.UUID;
12
13 @Service 3 usages
14 public class ProductService {
15
16     //Dependencies Injection
17     private final ProductRepository productRepo; 11 usages
18     public ProductService(ProductRepository productRepo) { no usages
19         this.productRepo = productRepo;
20     }
21
22     @Autowired 1 usage
23     private Path uploadsPath;
24
25 @  public Product addProduct(Product product, MultipartFile file) throws IOException { 1 usage
26     if (productRepo.existsByName(product.getName())) {
27         throw new RuntimeException("Product name already exists.");
28     }
29
30     if (file == null || file.isEmpty()) {
31         throw new RuntimeException("Please upload an image file.");
32     }
33
34     String fileName = UUID.randomUUID() + "_" + file.getOriginalFilename();
35     Path filePath = uploadsPath.resolve(fileName);
36     file.transferTo(filePath.toFile());
37
38     Product p = new Product(
39         product.getName(),
40         product.getPrice(),
41         product.getDescription()
42     );
43     p.setImageUrl("/images/" + fileName);
44
45     return productRepo.save(p);
46 }
47
```

- Testing in postman



- Show image in web browser (config)



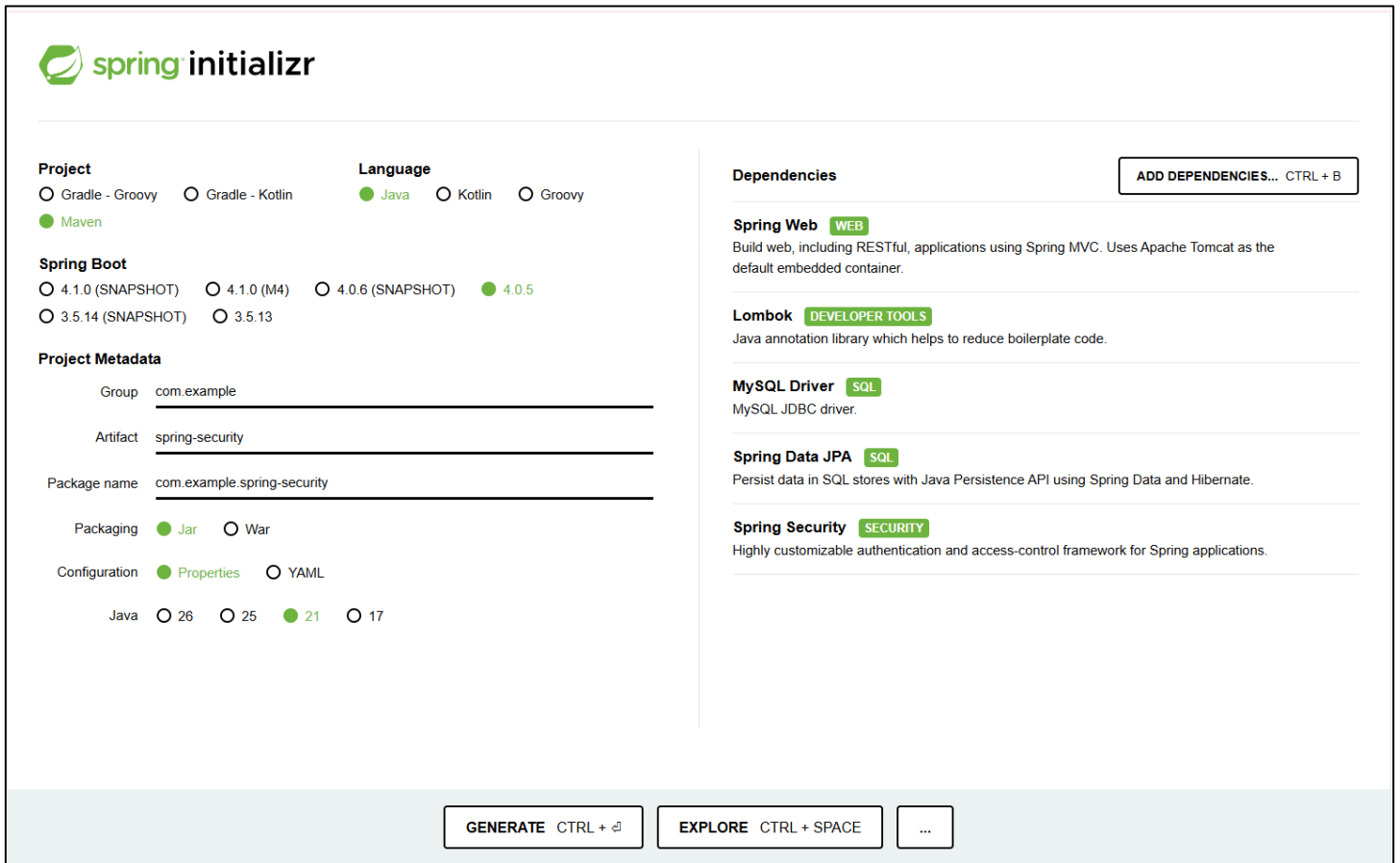
VII. Spring Security

- តើអ្វីទៅជា Security?

Security គឺជាការការពារ Application និង Data ពីការចូលប្រើដោយមិនត្រឹមត្រូវ (Unauthorized Access) និងការវាយប្រហារ (Attack)។ នៅក្នុង Web Application សុវត្ថិភាពមានតួនាទីសំខាន់ណាស់ ដើម្បីធានាថា:

- . ទិន្នន័យរបស់អ្នកប្រើប្រាស់មានសុវត្ថិភាព
- . គ្មានអ្នកចូលប្រើដោយគ្មានសិទ្ធិ
- . ប្រព័ន្ធដំណើរការយ៉ាងត្រឹមត្រូវ

- Project Set up spring security



The image shows the Spring Initializr web application interface for configuring a new project. The interface is divided into several sections:

- Project:** Includes radio buttons for **Gradle - Groovy**, **Gradle - Kotlin**, **Maven** (selected), **Language** (Java, Kotlin, Groovy), and **Spring Boot** versions (4.1.0 (SNAPSHOT), 4.1.0 (M4), 4.0.6 (SNAPSHOT), 4.0.5 (selected), 3.5.14 (SNAPSHOT), 3.5.13).
- Project Metadata:** Includes input fields for **Group** (com.example), **Artifact** (spring-security), and **Package name** (com.example.spring-security). It also has radio buttons for **Packaging** (Jar, War) and **Configuration** (Properties, YAML), and a **Java** version selector (26, 25, 21 (selected), 17).
- Dependencies:** A list of dependencies with checkboxes: **Spring Web** (WEB), **Lombok** (DEVELOPER TOOLS), **MySQL Driver** (SQL), **Spring Data JPA** (SQL), and **Spring Security** (SECURITY). An **ADD DEPENDENCIES...** button is also present.

At the bottom, there are buttons for **GENERATE** (CTRL + G), **EXPLORE** (CTRL + SPACE), and a **...** button.

- តើអ្វីជា Authentication?

Authentication គឺជា ការផ្ទៀងផ្ទាត់អត្តសញ្ញាណរបស់ user រា
ដំណើរការដើម្បីបញ្ជាក់ថាអ្នកជានរណា? ពេលគឺវាជាការLogin របស់ user។

- តើអ្វីជា Authorization?

Authorization គឺជាការកំណត់សិទ្ធិក្នុងការប្រើប្រាស់របស់userការត្រួត
ពិនិត្យថា “អ្នកអាចធ្វើអ្វីបាន?”.

- Connect to MySql

```
m pom.xml (spring-security)  SpringSecurityApplication.java  application.properties x
1  spring.application.name=spring-security
2
3  spring.datasource.url=jdbc:mysql://localhost:3306/spring-security
4  spring.datasource.username=root
5  spring.datasource.password=
6  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
7
8  spring.jpa.hibernate.ddl-auto=update
9  spring.jpa.show-sql=true
```

- Create table User and table role

```
m pom.xml (spring-security)  SpringSecurityApplication.java  application.properties  Role.java x
1  package com.example.spring_security.entity;
2
3  import jakarta.persistence.*;
4  import lombok.AllArgsConstructor;
5  import lombok.Builder;
6  import lombok.Data;
7  import lombok.NoArgsConstructor;
8
9  @Entity no usages
10 @Data
11 @AllArgsConstructor
12 @NoArgsConstructor
13 @Builder
14 @Table(name = "tbl_role")
15 public class Role {
16
17     @Id no usages
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     private String name; no usages
22
23 }
```



```
m pom.xml (spring-security)  SpringSecurityApplication.java  application.properties  Role.java  User.java x
1 package com.example.spring_security.entity;
2
3 import jakarta.persistence.*;
4 import lombok.AllArgsConstructor;
5 import lombok.Builder;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8
9 @AllArgsConstructor 1 usage
10 @NoArgsConstructor
11 @Data
12 @Builder
13 @Entity
14 @Table(name = "tbl_user")
15 public class User {
16
17     @Id no usages
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     @ManyToOne no usages
22     @JoinColumn(name = "role_id")
23     private Role role;
24     private String username; no usages
25     private String email; no usages
26     private String password; no usages
27 }
```

- Create role and user repository for manage query from db

```
SpringSecurityApplication.java  application.properties  entity\Role.java  User.java  RoleRepository.java x  UserRepository.java
1 package com.example.spring_security.repository;
2
3 import com.example.spring_security.entity.Role;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository no usages
8 public interface RoleRepository extends JpaRepository<Role, Long> { }
9
```

```
SpringSecurityApplication.java  application.properties  entity\Role.java  User.java  RoleRepository.java  UserRepository.java x
1 package com.example.spring_security.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5
6 @Repository 1 usage
7 public interface UserRepository extends JpaRepository<UserRepository, Long> { }
8
```



- Disable csrf ព្រោះ default spring security មាន csrf token

```

2026-04-02T07:09:46.301+07:00 INFO 16720 --- [spring-security] [      main] org.hibernate.orm.core           : HHH000489: No JTA platf
Hibernate: create table tbl_role (id bigint not null auto_increment, name varchar(255), primary key (id)) engine=InnoDB
Hibernate: create table tbl_user (id bigint not null auto_increment, email varchar(255), password varchar(255), username varchar(255), role_id begin
Hibernate: alter table tbl_user add constraint FKqyhp9ytkc0o8uapy1vtqmw350 foreign key (role_id) references tbl_role (id)
2026-04-02T07:09:46.496+07:00 INFO 16720 --- [spring-security] [      main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityM
2026-04-02T07:09:46.582+07:00 WARN 16720 --- [spring-security] [      main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view
2026-04-02T07:09:47.042+07:00 WARN 16720 --- [spring-security] [      main] .s.a.UserDetailsServiceAutoConfiguration :

Using generated security password: 50376d7e-f73c-4105-872e-1d6135575a09

This generated password is for development use only. Your security configuration must be updated before running your application in production.

```

```

tion.properties  Role.java  User.java  RoleRepository.java  UserRepository.java  SecurityConfig.java  CustomUserService.java
1  package com.example.spring_security.config;
2
3  import org.springframework.context.annotation.Bean;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6  import org.springframework.security.web.SecurityFilterChain;
7
8  @Configuration
9  public class SecurityConfig {
10
11      @Bean
12      @Autowired
13      public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
14          return http
15              .csrf(CsrfConfigurer<HttpSecurity> csrf -> csrf.disable()) // disable CSRF
16              .authorizeHttpRequests(AuthorizationManagerRequestMat... auth -> auth
17                  .anyRequest().permitAll() // allow all request
18              )
19              .httpBasic(HttpBasicConfigurer<HttpSecurity> basic->basic.disable())
20              .build();
21      }
22  }

```

→ បន្ទាប់ពីបានបង្កើត bean ដើម្បី disable csrf token ក្នុង spring security រួចរាល់ហើយក៏ដោយក៏វាទៅតែមិនទាន់អាចបិទ csrf token នោះបានដែរដោយសារតែយើងមិនទាន់មានទិន្នន័យ user ដែល load ពី database ដូច្នេះវាទៅតែ generate ជាធម្មតា ដើម្បីឱ្យអាចបិទ csrf បាន 100% ត្រូវបង្កើត class CustomerUserService ដើម្បីធ្វើការ load user ពី database ទើបវា disable បាន។

- Create CustomerUserService

```
1 package com.example.spring_security.auth;
2
3 > import ...
4
5 @Component no usages
6 public class CustomUserService implements UserDetailsService {
7
8     @Autowired 1 usage
9     private UserRepository userRepository;
10
11     @Autowired 1 usage
12     private RoleRepository roleRepository;
13
14     @Override no usages
15     public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
16         com.example.spring_security.entity.User user = userRepository.findByEmail(email).orElseThrow(
17             ()->new RuntimeException("user not found!"));
18         Role role = roleRepository.findByName("ROLE_USER").orElseThrow(
19             ()->new RuntimeException("role not found!"));
20
21         return User.builder()
22             .username(user.getEmail())
23             .password(user.getPassword())
24             .authorities(role.getName())
25             .build();
26     }
27 }
```

- Create user register and encrypt password

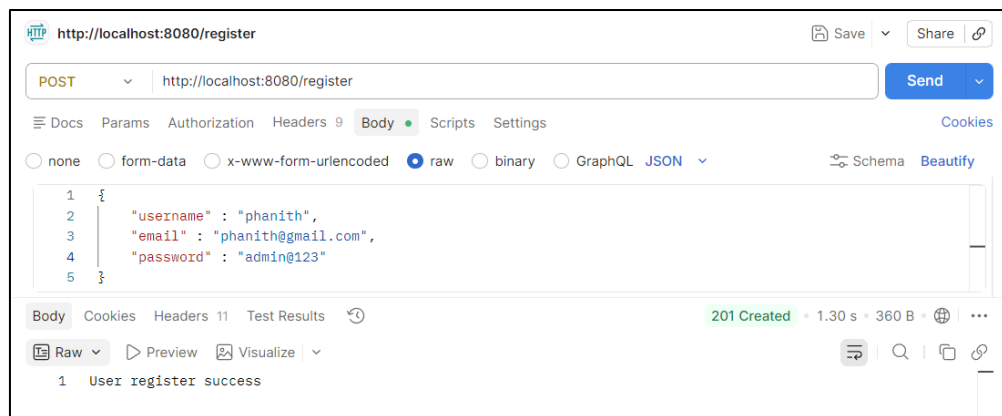
```
12 @Service no usages
13 public class UserAuthService {
14
15     @Autowired 2 usages
16     private UserRepository userRepo;
17     private RoleRepository roleRepo; 1 usage
18     private PasswordEncoder encoder; 1 usage
19
20     @PostMapping
21     public ResponseEntity<String> register(User user){ no usages
22         //check exists user by email
23         boolean exists = userRepo.existsByEmail(user.getEmail());
24         if(exists){
25             throw new RuntimeException("user email is already exists");
26         }
27         Role findRole = roleRepo.findByName("ROLE_USER").orElseThrow(
28             () -> new RuntimeException("Role not found!"));
29
30         User newUser = new User();
31         newUser.setEmail(user.getEmail());
32         newUser.setUsername(user.getUsername());
33         newUser.setRole(findRole);
34         newUser.setPassword(encoder.encode(user.getPassword()));
35
36         //save to db
37         User saved = userRepo.save(newUser);
38         return ResponseEntity.status(HttpStatus.CREATED).body("User register success");
39     }
```

- Create UserAuthController to work on end-point

```

1 package com.example.spring_security.controller;
2
3 > import ...
10 @RestController no usages
11 public class UserAuthController {
12
13     @Autowired 1 usage
14     private UserAuthService userService;
15
16     @PostMapping("/register") no usages
17     public ResponseEntity<String> addRegister(@RequestBody User user){
18         return userService.register(user);
19     }
20
21 }

```



Extra options

	id	email	password	username	role_id
☐ Edit Copy Delete	1	phanith@gmail.com	\$2a\$10\$taEzWCfHO3aVcJiGNxHeY.Ez0C8vw.YxEVb5L.nTaJK...	phanith	1
☐ Edit Copy Delete	2	channen@gmail.com	\$2a\$10\$IXznyrbe.SpeA6d6AlfoZ.nlhT398XbQuQ/StcvEf0d...	channen	2
☐ Edit Copy Delete	3	anna@gmail.com	\$2a\$10\$SrFallCdP8bDSesRqZGZjoO1rfn5Ql50Y4pouCNHFoZU...	anna	2

☐ Check all
 With selected:
 ☐ Edit
 ☐ Copy
 ☐ Delete
 ☐ Export



- Create user login

```
47 public String login(String email,String password){ no usages
48 // is a object to store user credential for submit to authenticationManager
49 UsernamePasswordAuthenticationToken authToken =
50 new UsernamePasswordAuthenticationToken(email, password);
51
52 try {
53 Authentication authentication = authenticationManager.authenticate(authToken);
54
55 if (authentication.isAuthenticated()) {
56 // make spring remember this user to access every end-point follow his role
57 SecurityContextHolder.getContext().setAuthentication(authentication);
58 request.getSession(b: true).setAttribute(
59 s: "SPRING_SECURITY_CONTEXT",
60 SecurityContextHolder.getContext());
61
62 return "Login Success";
63 } else {
64 return "Something wrong on email or password";
65 }
66 } catch (BadCredentialsException ex) {
67 return "Something wrong on email or password";
68 }
69 }
```

```
20
21 @PostMapping("/login") no usages
22 public String login(@RequestParam String email, @RequestParam String password){
23 return userService.login(email, password);
24 }
25 }
26 }
```

HTTP client interface showing a successful POST request to `http://localhost:8080/api/user/login` with parameters `email=anna@gmail.com` and `password=anan@123`. The response is `200 OK` with the body `Login Success`.

Key	Value	Description	Bulk Edit
☒ email	anna@gmail.com		
☒ password	anan@123		

Body: Cookies 1 Headers 11 Test Results 200 OK • 1.10 s • 347 B • ...

1 Login Success

- កំណត់ end-pointណាដែល user និងadmin អាចប្រើបាន

```
ProductController.java
1 package com.example.spring_security.controller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RequestMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 @RestController no usages
8 @RequestMapping("/api/product")
9 public class ProductController {
10
11     @GetMapping no usages
12     public String userAccess(){
13         return "Here is user page";
14     }
15
16     @GetMapping("/dashboard") no usages
17     public String adminAccess(){
18         return "Welcome to dashboard";
19     }
20
21 }
```

```
SecurityConfig.java
1 package com.example.spring_security.config;
2
3 > import ...
13 @Configuration no usages
14 public class SecurityConfig {
15
16     @Bean no usages
17     @
18     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
19         return http
20             .csrf(CsrConfigurer<HttpSecurity> csrf -> csrf.disable())
21             .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
22                 // Public API
23                 .requestMatchers(
24                     HttpMethod.POST,
25                     ...patterns: "/api/user/register",
26                     "/api/user/login"
27                 ).permitAll() AuthorizationManagerRequestMat...
28                 .requestMatchers(HttpMethod.GET, ...patterns: "/api/product") AuthorizedUr
29                 .hasAnyRole( ...roles: "USER", "ADMIN") AuthorizationManagerRequestMat...
30                 .requestMatchers(HttpMethod.GET, ...patterns: "/api/product/dashboard")
31                 .hasRole("ADMIN") AuthorizationManagerRequestMat...
32
33                 // Other request
34                 .anyRequest().authenticated()
35             )
36             .httpBasic(Customizer.withDefaults())
37             .build();
38
39     @Bean no usages
40     public PasswordEncoder encoder() {return new BCryptPasswordEncoder();}
41
42     @Bean no usages
43     @
44     public AuthenticationManager authenticationManager(AuthenticationConfiguration configura
45         return configuration.getAuthenticationManager();
46     }
```

